Reviewer Pack — Matroid Rank and ABP Size for Polynomial Representations

Entry d2edbef09287 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 02:20:19 UTC

Matroid Rank and ABP Size for Polynomial Representations

Entry ID: `d2edbef09287` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 02:20:19 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Algebraic Branching Program Size

Statement:

> The matroid rank of the coefficient matrix of a polynomial represented by an ABP of size s is Θ(log s).

Rationale (proposer's reasoning):

> Matroid rank captures the linear independence structure of the polynomial's coefficients, which may reflect the minimal complexity of representing the polynomial via an ABP. This bridge could expose hidden algebraic dependencies in the ABP's structure.

Taxonomy category: `BARRINGTON_ALG` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `091438f65c392c3e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i + max(range(i, m), key=lambda x: abs(A[x][i]))
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        if pivot == 0:
            continue
        for j in range(n):
            A[i][j] /= pivot
        for k in range(m):
            if k != i:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]

def matrix_rank(A):
    m, n = len(A), len(A[0])
    rank = 0
    for i in range(m):
        if any(A[i]):
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_rank = 0
    instances_tested = 0

    for n in n_values:
        for _ in range(5):  # Ensure at least 5 instances per size
            coefficients = [random.randint(-10, 10) for _ in range(n + 1)]
            A = [[coefficients[i] * j**k for k in range(n + 1)] for i in range(n + 1)]
            gaussian_elimination(A)
            rank = matrix_rank(A)
            total_rank += rank
            instances_tested += 1

    mean_rank = total_rank / instances_tested
    conjecture_holds = math.isclose(mean_rank, math.log(instances_tested), rel_tol=0.2)
```

```
    counterexample = "" if conjecture_holds else f"mean_rank={mean_rank}, expected_log={math.log(i

    return {
        "metric_name": "Matroid Rank",
        "metric_value": mean_rank,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b11b003.py", line 75, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2b11b003.py", line 51, in run_trial
    A = [[coefficients[i] * j**k for k in range(n + 1)] for i in range(n + 1)]
                            ^
NameError: name 'j' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to undefined variable 'j', preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Fix the NameError in test_2b11b003.py by defining variable 'j' in the list comprehension

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	135712
2	propose	ollama_remote	qwen3:8b	0	0	136306
3	preregistration	ollama_remote	qwen3:8b	0	0	31467
4	novelty	ollama_remote	qwen3:8b	0	0	27545
5	novelty	ollama_remote	qwen3:8b	0	0	18952
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11676
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9906
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10955
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8880
10	judge	ollama_remote	qwen3:8b	0	0	22654

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 414053 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/d2edbef09287.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d2edbef09287.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/d2edbef09287.tar.gz` (if generated)